

Changing Target Object Class for Adversarial Attacks

Complete Guide: From Person Detection to Any Object Class

This guide explains how to modify AdvReal to attack different object classes (e.g., attacking cars instead of people). Whether you're targeting vehicles, animals, or custom objects, this guide covers all necessary configuration changes.

Quick Overview

What you need to change:

1.  Update **ATTACK_CLASS** in configuration file
 2.  Verify class ID in names file
 3.  Fix hardcoded extractors (if using train.py)
 4.  Update 3D assets (for realistic rendering)
-

Configuration: The Single Source of Truth

ATTACK_CLASS Parameter

The target class is controlled by a **single parameter** in your YAML configuration:

```
ATTACKER:  
  ATTACK_CLASS: '0' #  This controls which object to attack
```

Where it's used:

-  **Parsing:** `utils/parser.py` → `ConfigParser` reads and stores it
 -  **Filtering:** `attack/attacker.py` → Filters detections by class
 -  **Loss computation:** Probability extractors use it to select target boxes
-

Step-by-Step: Find Your Target Class ID

Method 1: Check the Names File

Default COCO 80 classes: `configs/namefiles/coco80.names`

```
# View class names with line numbers  
cat -n configs/namefiles/coco80.names
```

Common COCO class IDs:

Class	ID	Class	ID	Class	ID	Class	ID	Class	ID	Class	ID	Class	ID	Class	ID	Class	ID	Class	ID	Class	ID								
person	0	bicycle	1	car	2	motorcycle	3	airplane	4	bus	5	train	6	truck	7	boat	8	cat	15	dog	16	horse	17	elephant	20	bear	21	zebra	22

Method 2: Programmatic Lookup

```
from utils.parser import load_class_names

# Load class names
classes = load_class_names('configs/namefiles/coco80.names')

# Find class ID
target_class = 'car'
class_id = classes.index(target_class)
print(f"✅ '{target_class}' has ID: {class_id}")

# Output: ✅ 'car' has ID: 2
```

 Configuration Update Tutorial **Example: Attack Cars Instead of People****Before (Person attack):**

```
ATTACKER:
  ATTACK_CLASS: '0' # Person
```

After (Car attack):

```
ATTACKER:
  ATTACK_CLASS: '2' # Car
```

 Complete Configuration Example**File:** configs/baseline/v5_car.yaml

```
# Dataset Configuration
DATA:
  CLASS_NAME_FILE: 'configs/namefiles/coco80.names' # 📁 Verify this path
  AUGMENT: 0
```

```

TRAIN:
  IMG_DIR: 'data/car_images/train' # 📁 Use car-specific dataset
  LAB_DIR: 'data/car_images/labels'

# Detector Settings
DETECTOR:
  NAME: ["YOLOV5"]
  INPUT_SIZE: [416, 416]
  BATCH_SIZE: 8
  CONF_THRESH: 0.5
  IOU_THRESH: 0.45

# Attack Configuration
ATTACKER:
  METHOD: "optim"
  EPSILON: 255
  MAX_EPOCH: 800
  STEP_LR: 0.03
  ATTACK_CLASS: '2' # 📁 Car class ID
  LOSS_FUNC: "obj-tv"
  tv_eta: 2.5

PATCH:
  WIDTH: 300
  HEIGHT: 300
  SCALE: 0.15
  INIT: "gray"
  TRANSFORM: ['jitter', 'rotate', 'median_pool']

```

⚠ Critical Issue: Hardcoded Extractors

The Problem

Some probability extractors in `load_data.py` **hardcode** `attack_cls = 0`, ignoring the configuration:

```

# ✗ WRONG: Hardcoded person class
class YOLOv2MaxProbExtractor(nn.Module):
    def forward(self, YOLOoutputs, gt, loss_type, iou_thresh):
        # ...
        attack_cls = 0 # 📁 Always attacks person!
        mask = mask & (cls_idx == attack_cls)

```

Impact: Even if you change `ATTACK_CLASS` in config, training still targets person class!

The Solution: Fix Extractors

✅ Correct Pattern (from YOLOv11)

```
class YOLOv11MaxProbExtractor(nn.Module):
    def __init__(self, cls_id, num_cls, model, figsize):
        super().__init__()
        self.cls_id = cls_id
        self.num_cls = num_cls
        self.figsize = figsize
        self.model = model
        self.cfg = None # 📁 Will be set externally

    def forward(self, YOLOoutputs, gt, loss_type, iou_thresh):
        # ...

        # ✅ CORRECT: Read from config
        from easydict import EasyDict
        attack_cls = int(getattr(
            self.cfg,
            'ATTACKER',
            EasyDict(ATTACK_CLASS='0')
        ).ATTACK_CLASS)

        mask = mask & (cls_idx == attack_cls)
        # ...
```

🔧 Files That Need Fixing

File	Class	Status
load_data.py	YOLOv2MaxProbExtractor	❌ Needs fix
load_data.py	YOLOv5MaxProbExtractor	❌ Needs fix
load_data.py	YOLOv11MaxProbExtractor	✅ Already correct
load_data.py	MaxProbExtractor (Faster R-CNN)	⚠️ Uses cls_id parameter

🔧 How to Fix YOLOv2/YOLOv5 Extractors

Step 1: Add self.cfg attribute in __init__:

```
class YOLOv2MaxProbExtractor(nn.Module):
    def __init__(self, cls_id, num_cls, model, figsize):
        super().__init__()
        self.cls_id = cls_id
        self.num_cls = num_cls
        self.figsize = figsize
        self.model = model
        self.cfg = None # 📁 Add this line
```

Step 2: Read attack class from config in forward():

```
def forward(self, YOLOoutputs, gt, loss_type, iou_thresh):
    # ... existing code ...

    # Replace: attack_cls = 0
    # With:
    from easydict import EasyDict
    attack_cls = int(getattr(
        self.cfg,
        'ATTACKER',
        EasyDict(ATTACK_CLASS='0')
    ).ATTACK_CLASS)

    mask = ious.ge(iou_thresh) & (boxes[..., 6] == attack_cls)
    # ... rest of code ...
```

Step 3: Set config after creating extractor in `train.py`:

```
# In PatchTrainer.__init__()
if args.arch == "yolov2":
    self.probab_extractor = YOLOv2MaxProbExtractor(
        cls_id=0, num_cls=80, model=self.model, figsize=self.img_size
    )
    self.probab_extractor.cfg = cfg # Add this line
```

☒ Two Training Paths: Understanding the Difference

AdvReal has **two distinct attack pipelines**:

Path 1: Multi-Detector Attack Framework

File: `attack/attacker.py` → `UniversalAttacker`

```
# ☑ This path RESPECTS ATTACK_CLASS automatically
detector_attacker = UniversalAttacker(cfg, device)
detector_attacker.attack(img_batch) # Uses cfg.attack_cls internally
```

Characteristics:

- ☑ Reads `ATTACK_CLASS` from config correctly
- ☑ Works with multiple detectors
- ☑ No extractor modification needed
- ☑ Used for adversarial evaluation

Use case: Evaluating patch effectiveness across multiple models

Path 2: 3D Patch Training Pipeline

File: `train.py` → `PatchTrainer`

```
# ⚠ This path uses model-specific extractors
trainer = PatchTrainer(args)
trainer.train() # Uses prob_extractor internally
```

Characteristics:

- ⚠ Uses probability extractors from `load_data.py`
- ⚠ Some extractors hardcode `attack_cls = 0`
- ⚠ Requires manual fixes (see above)
- 🛠 Enables 3D rendering and NRSM

Use case: Training patches with realistic 3D rendering



Complete Checklist

Use this checklist when changing target class:



Configuration Changes

- ☐ **Find class ID** in `configs/namefiles/coco80.names`
- ☐ **Update ATTACK_CLASS** in your config YAML (e.g., '0' → '2')
- ☐ **Verify class names file** path in `DATA.CLASS_NAME_FILE`
- ☐ **Update dataset path** to match new object class



Code Fixes (for train.py path)

- ☐ **Check extractor** for your architecture (YOLOv2/v5/v11)
- ☐ **Add `self.cfg`** attribute if missing
- ☐ **Replace hardcoded 0** with config lookup
- ☐ **Set extractor.cfg** in `train.py`



Asset Updates (for 3D rendering)

- ☐ **3D mesh files** matching new object (e.g., car.obj)
- ☐ **Texture coordinates** (UV maps)
- ☐ **Training images** containing target class
- ☐ **Camera parameters** (distance, angle) appropriate for object



Validation

- ☐ **Run quick test** with 10 epochs
- ☐ **Inspect bbox_array** output for correct class IDs
- ☐ **Monitor loss values** (should decrease)
- ☐ **Visual check** on rendered images

Example: Complete Car Attack Setup

 Goal: Attack YOLOv5 on Cars

Step 1: Configuration

```
# Create new config
cp configs/baseline/v5.yaml configs/baseline/v5_car.yaml
```

```
# Edit v5_car.yaml
ATTACKER:
  ATTACK_CLASS: '2' # Car class
```

Step 2: Fix Extractor (if needed)

```
# In load_data.py → YOLOv5MaxProbExtractor
def __init__(self, cls_id, num_cls, model, figsize):
    # ... existing code ...
    self.cfg = None # Add this

def forward(self, YOLOoutputs, gt, loss_type, iou_thresh):
    # ... existing code ...

    # Replace: attack_cls = 0
    attack_cls = int(getattr(
        self.cfg, 'ATTACKER',
        EasyDict(ATTACK_CLASS='0')
    ).ATTACK_CLASS)
```

Step 3: Train

```
python train.py \
  --nepoch 800 \
  --save_path results/yolov5_car_patch \
  --arch yolov5 \
  --cfg configs/baseline/v5_car.yaml \
  --seed_type fixed \
  --loss_type max_iou \
  --lr 0.03
```

Step 4: Verify

```
# Quick verification script
import torch
from utils.parser import ConfigParser

cfg = ConfigParser("configs/baseline/v5_car.yaml")
print(f"✅ Attacking class: {cfg.attack_cls}") # Should be 2
print(f"✅ Class name: {cfg.all_class_names[cfg.attack_cls]}") # Should be 'car'
```

🧠 Advanced: 3D Assets for New Target Objects

When attacking non-person objects with 3D rendering, you need appropriate assets.

📦 Required Assets

Asset Type	Purpose	Example (Person)	Example (Car)
3D Mesh	Object geometry	man.obj	sedan.obj
Texture UV	Patch placement	T-shirt UV map	Car hood UV map
Training Data	Ground truth	INRIA pedestrians	COCO cars
Background Images	Scene compositing	Street scenes	Roads/parking lots

🔗 Mesh Preparation Workflow

1. Acquire 3D Model

Sources:

-  [Sketchfab](#) (filter: downloadable, free)
-  [TurboSquid Free Models](#)
- 🧠 Create custom with Blender

Requirements:

-  Closed, watertight mesh
-  Reasonable polygon count (5K-50K triangles)
-  Clean UV unwrapping

2. UV Map for Patch Placement

Using Blender:

```
# 1. Import model: File → Import → Wavefront (.obj)
# 2. Select object, switch to Edit Mode (Tab)
# 3. Select faces where patch should appear (e.g., car hood)
# 4. U → Unwrap → Smart UV Project
```

```
# 5. UV Editor → Export UV Layout
# 6. File → Export → Wavefront (.obj)
```

Export settings:

- ☒ Include UVs
- ☒ Triangulate faces
- ☒ Write normals
- ☒ Y-axis up

3. Integrate into AdvReal

File locations:

```
data/Archive/
├── car_model/
│   ├── car.obj          # Main geometry
│   ├── car.mtl          # Material properties
│   └── textures/
│       └── default.png  # Base texture
```

Update renderer in `train.py`:

```
class PatchTrainer(object):
    def __init__(self, args):
        # ... existing code ...

        # Load 3D meshes for target object
        if args.target_object == "car":
            mesh_car = "data/Archive/car_model/car.obj"
            self.mesh_obj = load_objs_as_meshes(
                [mesh_car],
                device=self.device
            )
```

 Camera & Rendering Parameters

Different objects require different camera setups:

Object	Distance	Elevation	FOV	Scale
Person	5-10m	Eye level ($\pm 10^\circ$)	45°	1.0
Car	10-20m	Slightly above (+20°)	50°	1.5
Animal	3-8m	Ground level (0°)	40°	0.8

Object	Distance	Elevation	FOV	Scale
Drone	15-30m	Below (-30°)	60°	0.5

Update in **render.py**:

```
# Camera positioning for car rendering
R, T = look_at_view_transform(
    dist=15.0,          # Distance from object
    elev=20.0,          # Elevation angle (degrees)
    azim=np.random.uniform(-30, 30) # Random azimuth
)

cameras = FoVPerspectiveCameras(
    device=device,
    R=R, T=T,
    fov=50.0           # Field of view
)
```

Dataset Preparation

Required structure:

```
data/
├── car_images/
│   ├── train/
│   │   ├── 000001.jpg
│   │   ├── 000002.jpg
│   │   └── ...
│   └── labels/
│       ├── 000001.txt # YOLO format: class x y w h
│       ├── 000002.txt
│       └── ...
└── background_car/
    └── highway_scenes/
        ├── bg_001.jpg
        ├── bg_002.jpg
        └── ...
```

Label format (YOLO):

```
# 000001.txt (normalized coordinates)
2 0.456 0.678 0.234 0.187 # class_id x_center y_center width height
```

Testing & Validation

Quick Sanity Check

```
# test_target_class.py
import torch
from utils.parser import ConfigParser
from attack.attacker import UniversalAttacker

# Load config
cfg = ConfigParser("configs/baseline/v5_car.yaml")

print("=" * 50)
print("🌀 Target Class Configuration")
print("=" * 50)
print(f"ATTACK_CLASS (config): {cfg.ATTACKER.ATTACK_CLASS}")
print(f"attack_cls (parsed): {cfg.attack_cls}")
print(f"Class name: {cfg.all_class_names[cfg.attack_cls]}")
print(f"Attack list: {cfg.attack_list}")
print("=" * 50)

# Initialize attacker
attacker = UniversalAttacker(cfg, torch.device('cuda'))
print(f"\n✅ Attacker initialized successfully!")
print(f"Target class filter: {attacker.cfg.attack_cls}")
```

Expected output:

```
=====
🌀 Target Class Configuration
=====
ATTACK_CLASS (config): 2
attack_cls (parsed): 2
Class name: car
Attack list: [2]
=====

✅ Attacker initialized successfully!
Target class filter: 2
```

Visual Validation

```
# visualize_detections.py
import cv2
import torch
from detlib.utils import init_detector
from utils.parser import ConfigParser
from utils.det_utils import plot_boxes_cv2
```

```
cfg = ConfigParser("configs/baseline/v5_car.yaml")
detector = init_detector("yolov5", cfg.DETECTOR)

# Load test image
img = cv2.imread("data/test_images/car_sample.jpg")
img_tensor = torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0).float() / 255.0
img_tensor = img_tensor.cuda()

# Run detection
output = detector(img_tensor)
boxes = output['bbox_array'][0]

# Filter by target class
target_class_id = cfg.attack_cls
target_boxes = boxes[boxes[:, 5] == target_class_id]

print(f"🔍 Found {len(target_boxes)} {cfg.all_class_names[target_class_id]}(s)")

# Visualize
result = plot_boxes_cv2(img, target_boxes.cpu(), cfg.all_class_names)
cv2.imwrite("detection_result.jpg", result)
print("✅ Saved visualization to detection_result.jpg")
```

Troubleshooting

Problem	Possible Cause	Solution
Loss not decreasing	Wrong class ID	Verify class index in names file
No detections	Dataset mismatch	Ensure images contain target class
Still attacks person	Hardcoded extractor	Fix <code>load_data.py</code> as described above
Mesh rendering fails	Invalid OBJ file	Check mesh normals and UV maps
Memory error	3D mesh too large	Decimate mesh or reduce batch size
Patch not visible	Wrong UV coordinates	Re-unwrap mesh in Blender

Additional Resources

External Tools

- **3D Modeling:** [Blender](#) (free, open-source)
- **Mesh Repair:** [MeshLab](#)
- **UV Editing:** [RizomUV](#) (has free version)
- **Dataset Annotation:** [CVAT](#), [LabellImg](#)

Documentation

- [COCO Dataset Classes](#)

- [PyTorch3D Tutorials](#)
 - [YOLO Label Format](#)
-

Summary

Minimum steps to change target class:

1. ☒ **Find class ID** in `configs/namefiles/coco80.names`
2. ☒ **Update config:** `ATTACK_CLASS: '2'` (for car)
3. ☒ **Fix extractors** (if using `train.py` with YOLOv2/v5)
4. ☒ **Test with validation script**

For complete 3D pipeline:

5. ☒ **Prepare 3D mesh** with proper UV mapping
 6. ☒ **Update dataset** with target class images
 7. ☒ **Adjust camera** parameters for object scale
 8. ☒ **Validate rendering** visually
-

 **You're ready to attack any object class!**

Still having issues? Check:

- ☒ Extractor is reading from `self.cfg` (not hardcoded)
- ☒ Config file has correct `ATTACK_CLASS` value
- ☒ Dataset contains target class annotations
- ☒ 3D mesh has valid UV coordinates (for rendering path)